

```
from queue import PriorityQueue
```

```
def best_first_search(graph, start, target, heuristics):
```

```
    # Create a priority queue to store the nodes to be explored
```

```
    queue = PriorityQueue()
```

```
    # FIX 1 & 2: Match the 3-tuple structure and initialize the path with [start]
```

```
    queue.put((heuristics[start], start, [start])) # (priority, node, path)
```

```
    # Create a set to keep track of visited nodes
```

```
    visited = set()
```

```
    while not queue.empty():
```

```
        # Get the node with the lowest heuristic cost
```

```
        h, node, path = queue.get()
```

```
        # If we reached the goal, return the path
```

```
        if node == target:
```

```
            return path, h
```

```
        # FIX 3: Mark visited ONLY when popping from the queue, not when pushing.
```

```
        if node in visited:
```

```
            continue
```

```
        visited.add(node)
```

```
        # Explore neighbors
```

```
        for neighbor in graph.get(node, []):
```

```
            if neighbor not in visited:
```

```
                # Add neighbor to the queue with updated path
```

```
queue.put((heuristics[neighbor], neighbor, path + [neighbor]))
```

```
return None, None # Return None if no path is found
```

```
# Example usage
```

```
if __name__ == "__main__":
```

```
graph = {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['D', 'E'],
```

```
    'C': ['F'],
```

```
    'D': [],
```

```
    'E': [],
```

```
    'F': []
```

```
}
```

```
heuristics = {
```

```
    'A': 5,
```

```
    'B': 4,
```

```
    'C': 2,
```

```
    'D': 0,
```

```
    'E': 1,
```

```
    'F': 0
```

```
}
```

```
path, cost = best_first_search(graph, 'A', 'F', heuristics)
```

```
print(f"Path found by Best First Search: {path} with Target heuristic cost: {cost}")
```

```
print("\n-----")
```

```
print("Program executed successfully by:")
```

```
print("Name: Bhuvan K")
```

```
print("Roll No: [24BECS118]")
```

```
print("-----")
```



The screenshot shows a VS Code terminal window with the following content:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\91638\OneDrive\Documents\Ai> python -u "c:\Users\91638\OneDrive\Documents\Ai\BEST FIRST SEARCH.py"
● Path found by Best First Search: ['A', 'C', 'F'] with Target heuristic cost: 0

-----
Program executed successfully by:
Name: Bhuvan K
Roll No: [24BECS118]
-----
❖ PS C:\Users\91638\OneDrive\Documents\Ai>
```

The terminal window has a dark theme. The top bar shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The output shows the execution of a Python script that finds a path and prints the name and roll number. The status bar at the bottom indicates the current line and column (Ln 59, Col 50), the number of spaces (4), the encoding (UTF-8), the line ending (CRLF), the language (Python), and the version (3.14.2).